

Network Traffic Profiler

Capture, fingerprint, and classify web traffic — straight from your browser.

BSEF24A024 – SHAYAN GHAFAR

Contents

Overview	3
Architecture Diagram	4
• Static UI	4
• Flask API	4
• Capture Module	4
• Extraction Module.....	4
• Fingerprint & Classification	4
Flowchart LR.....	4
Backend Details.....	5
Main Python Modules.....	5
Pipeline Flow	5
1. Capture.....	5
2. Extract	5
3. Fingerprint.....	5
4. Enrich & Classify.....	5
Frontend Details.....	6
Static Files	6
UI Features	6
Development Steps.....	6
1. Scaffolding.....	6
2. Traffic Capture	6
3. Feature Extraction.....	6
4. Fingerprint Design.....	6
5. Classification Logic	7
6. Comparison Endpoint	7
7. UI Build.....	7
8. Testing	7

9. Polish	7
Key Metrics / Sample Output.....	7
Metric	7
Description	7
Sample Value	7
What I Learned.....	8
• Network capture in Python is deceptively nuanced —	8
• Data-driven design pays off	8
• Simple rule-based classifiers can be surprisingly effective	8
• Frontend-backend integration needs a contract from day one	8
• Performance tuning matters for live capture.....	8
• Fixture-based testing saves debugging time.....	8
Challenges & Solutions.....	8
Permissions for Packet Capture	8
Handling Large PCAP Files	8
Cross-Traffic Interference	9
Behavioral Label Issue.....	9

Overview

The **Network Traffic Profiler** is a web-based tool that captures live network traffic for any user-supplied URL, extracts meaningful features from the captured packets, and generates a deterministic fingerprint that can be used to classify and compare traffic patterns. It is designed to help security researchers, network engineers, and curious developers understand *what* a website sends over the wire — and *how* that signature compares to known traffic profiles. The system exposes a clean REST API (built with Flask) and a responsive single-page UI that makes the entire pipeline accessible without touching the command line.

Architecture Diagram

The application is organised into five logical layers that form a linear pipeline:

- **Static UI** — three vanilla files (index.html, style.css, script.js) served directly by Flask's static file handler.
- **Flask API** — the entry point (app.py) that routes requests, enforces input validation, and orchestrates the pipeline.
- **Capture Module** — uses Scapy to perform a controlled traffic capture while requests fetches the target URL.
- **Extraction Module** — parses the raw PCAP data and derives per-packet and per-flow features (byte counts, inter-arrival times, protocol distribution, etc.).
- **Fingerprint & Classification** — hashes the feature vector into a short hex fingerprint and applies rule-based thresholds to classify the traffic (e.g., *streaming*, *API call*, *web browsing*, *encrypted blob*).

Flowchart LR

UI[Static UI] --> API[Flask API]

API --> Capture[Capture Scapy]

Capture --> Extract[Extract Features]

Extract --> Fingerprint[Generate Fingerprint]

Fingerprint --> Classify[Enrich & Classify]

Data flows left-to-right; every stage returns a structured Python dict that is passed to the next stage, keeping each module independently testable.

Backend Details

Main Python Modules

<i>File</i>	<i>Responsibility</i>
app.py	Flask app factory, route definitions, CORS setup
capture.py	Scapy-based packet sniffer, bounded by timeout and packet count
extract.py	Feature extraction (bytes, IPs, ports, inter-arrival times, TLS hello detection)
fingerprint.py	Deterministic fingerprint from the sorted feature vector
classify.py	Rule-based classifier + optional external enrichment via IP geolocation

Pipeline Flow

1. **Capture** — `capture_traffic(url)` spins up a Scapy sniffer on the active interface, triggers an HTTP/S request via `requests.get(url)`, records all packets exchanged in the process, and writes a temporary .pcap file.
 2. **Extract** — `extract_features(packets)` iterates over the PCAP, groups packets into flows (5-tuple), and computes per-flow statistics: total bytes, packet count, mean/std packet size, flow duration, and protocol flags.
 3. **Fingerprint** — `fingerprint(features)` sorts the feature keys alphabetically, serialises the dict to JSON, and returns the first 16 hex characters of its SHA-256 digest — a stable, reproducible identifier.
 4. **Enrich & Classify** — `classify()` maps the fingerprint against a local rule database, tags each unique IP with its AS organisation (via ipwhois), and assigns a human-readable traffic category.
-

Frontend Details

Static Files

- **index.html** — single-page layout with a URL input form, a results panel, and a Chart.js doughnut chart for protocol distribution.
- **style.css** — custom design system using CSS custom properties (see snippet below), Flexbox grid, and subtle entrance animations.
- **script.js** — fetches /api/analyze on form submission, parses the JSON response, and progressively renders each section of the results panel without a full page reload.

UI Features

- **Responsive layout** — the results grid collapses from two columns to one below 640 px using a single CSS media query.
 - **Modern colour palette** — a warm, accessible palette defined as CSS variables (--color-bubu, --color-dudu, etc.) so every element can be re-themed from one place.
 - **Micro-animations** — the results panel fades and slides in via a @keyframes animation triggered by toggling a class, giving the UI a polished feel without JavaScript animation libraries.
-

Development Steps

1. **Scaffolding** — Initialised a Flask project with a static/ folder and a requirements.txt. Configured flask run for local development.
2. **Traffic Capture** — Integrated Scapy's AsyncSniffer to capture packets in a background thread while requests fetched the target URL in the main thread.
3. **Feature Extraction** — Built extract.py to iterate over PCAP packets and aggregate flow-level statistics into a flat dictionary.
4. **Fingerprint Design** — Chose a truncated SHA-256 of the sorted feature JSON so that identical traffic patterns always produce the same fingerprint, enabling comparison across sessions.

5. **Classification Logic** — Implemented a rule-based classifier with threshold conditions (e.g., mean packet size > 1400 bytes → likely *streaming*) stored in a YAML rule file for easy iteration.
 6. **Comparison Endpoint** — Added `/api/compare` to accept two fingerprints and return a Jaccard-style similarity score based on shared feature buckets.
 7. **UI Build** — Wrote the single-page frontend, wired up Chart.js for the doughnut chart, and styled everything with the Bubu-Dudu token system.
 8. **Testing** — Added pytest unit tests for `extract.py` and `fingerprint.py` using pre-captured .pcap fixtures. Wrote a simple curl-based smoke test for the API.
 9. **Polish** — Added input validation, meaningful error messages, CORS headers, rate-limiting via Flask-Limiter, and the micro-animations in the frontend.
-

Key Metrics / Sample Output

<u>Metric</u>	<u>Description</u>	<u>Sample Value</u>
total_bytes	Sum of all captured packet payloads	482,304 B
packet_count	Total number of packets in the capture	412
unique_ips	Number of distinct IP addresses observed	7
mean_packet_size	Average payload size across all packets	1,171 B
flow_count	Number of distinct 5-tuple flows	14
tls_ratio	Fraction of packets with a TLS Client Hello	0.68
capture_duration_s	Wall-clock time of the capture window	3.2 s
fingerprint	Truncated SHA-256 of the feature vector	a3f9c12b88e04d1c
traffic_class	Human-readable classification label	encrypted_web

What I Learned

- **Network capture in Python is deceptively nuanced** — Scapy requires elevated privileges, and choosing the right interface (especially on macOS) took more trial-and-error than expected.
 - **Data-driven design pays off** — storing classification rules in a YAML file rather than hard-coding them in Python made iterating on thresholds *much* faster and kept the logic easy to reason about.
 - **Simple rule-based classifiers can be surprisingly effective** — for a first pass, threshold rules on 4–5 features (packet size, TLS ratio, flow duration, byte volume) correctly classified the vast majority of test URLs.
 - **Frontend-backend integration needs a contract from day one** — defining the JSON response schema upfront prevented a lot of back-and-forth between app.py and script.js later.
 - **Performance tuning matters for live capture** — early versions blocked the Flask response thread during capture. Moving the sniffer to a background thread and using a timeout-bounded join cut perceived latency by ~60 %.
 - **Fixture-based testing saves debugging time** — saving real .pcap files as test fixtures meant I could iterate on extract.py logic without re-running live captures, making the test suite fast and deterministic.
-

Challenges & Solutions

Permissions for Packet Capture

Raw packet capture requires root (or CAP_NET_RAW) on Linux/macOS. Running the entire Flask app as root is a bad idea. **Solution:** Isolated the capture step into a small helper script (capture_helper.py) invoked via subprocess with sudo, using a tightly scoped sudoers rule. On Windows, Npcap was installed to grant the privilege to the Python process directly.

Handling Large PCAP Files

For URL targets with many redirects or heavy media content, captures could balloon to tens of megabytes, slowing down feature extraction. **Solution:** Added a hard cap of MAX_PACKETS =

2000 and TIMEOUT = 10s to capture.py, and streamed the PCAP through Scapy's PcapReader iterator instead of loading the whole file into memory.

Cross-Traffic Interference

On a busy development machine, captures picked up unrelated background traffic (OS updates, Slack, etc.), polluting the feature set. **Solution:** Applied a BPF filter scoped to the destination IPs returned by a pre-capture DNS lookup of the target URL — this reduced noise dramatically and made fingerprints significantly more stable across repeated captures of the same URL.

Behavioral Label Issue

The behavioral label issue was occurring, if I classify using a specific check then it misses some other web and vice versa. **Solution:** Took average and increased capture and hook time to reduce this error but not fully eradicate it.

Thank you for taking the time to read through this report! If you have questions, suggestions, or want to collaborate on any of the future enhancements, feel free to reach out.

— **Shayan Ghaffar** | bsef24a024@pucit.edu.pk | https://github.com/I-S-G-7/network_profiler